

Simulación de Ecosistema con OpenMP

Proyecto — Computación Paralela y Distribuida

Humberto Alexander de la Cruz — Carné 23735

Daniel Oswaldo Juárez Herrera - 23709

13 de agosto de 2026

1. Introducción

Se implementó una simulación de un ecosistema de tres especies (plantas, herbívoros y carnívoros) sobre una cuadrícula bidimensional, paralelizada con OpenMP. El sistema es un **autómata celular con agentes móviles**: el espacio se discretiza en celdas y el tiempo en *ticks*, y en cada tick todos los agentes actúan simultáneamente según reglas locales.

El objetivo del proyecto no es únicamente producir una simulación correcta, sino **paralelizarla sin alterar su resultado**. Como se explica en la Sección 3, la paralelización ingenua que sugiere el enunciado introduce condiciones de carrera y hace que la salida dependa del número de hilos. Resolver ese problema constituye la parte central del trabajo.

2. Reglas del ecosistema

2.1. Espacio y vecindario

El ecosistema es una cuadrícula de $W \times H$ celdas. Cada celda puede estar **vacía** o contener **exactamente una** entidad: una planta, un herbívoro o un carnívoro.

Se emplea el **vecindario de Moore**: los 8 vecinos que rodean a una celda, incluyendo las diagonales. Todas las interacciones son locales; ninguna entidad ve más allá de esas 8 celdas.

La frontera es **toroidal**: el borde derecho se conecta con el izquierdo y el superior con el inferior. Esta decisión elimina los casos especiales de borde y garantiza que *toda* celda tenga exactamente 8 vecinos, lo que a su vez uniformiza la regla de muerte por hacinamiento de las plantas.

2.2. Estado de una celda

Campo	Tipo	Significado
tipo	entero	Vacío, planta, herbívoro o carnívoro
energia	entero	Se acumula al comer; se gasta al reproducirse
edad	entero	Ticks vividos; determina la muerte por vejez
hambre	entero	Ticks <i>consecutivos</i> sin comer

2.3. Plantas

Las plantas son productores primarios: no se mueven ni consumen recursos.

- **Reproducción.** En cada tick, una planta intenta expandirse a una celda vecina *vacía* con probabilidad p_{repro} (por defecto 0,25). La celda destino se elige uniformemente al azar entre las vecinas vacías. La planta madre permanece en su lugar.
- **Movimiento.** Ninguno.
- **Muerte.** Por dos causas:

- (a) *Hacinamiento*: si sus 8 vecinos son plantas, muere por falta de espacio para crecer.
- (b) *Depredación*: si un herbívoro se mueve a su celda.

2.4. Herbívoros

En cada tick, un herbívoro evalúa su vecindario y elige una acción por **prioridad estricta**:

Prioridad	Condición	Acción
1	Hay un carnívoro adyacente	Huir : moverse a una celda vecina vacía elegida al azar
2	Hay una planta adyacente	Comer : moverse a esa celda y consumir la planta (+1 energía, hambre a 0)
3	Hay una celda vecina vacía	Explorar : moverse a ella
4	Ninguna de las anteriores	Permanecer inmóvil

Que huir tenga prioridad sobre comer es una decisión de diseño: la supervivencia inmediata pesa más que la alimentación. Si el herbívoro huye o explora, no come en ese tick.

- **Reproducción**. Un herbívoro que *se mueve* y cuya energía alcanza el umbral E_{repro} (por defecto 3) deja una cría en la celda que acaba de desocupar, y descuenta esa energía. La cría nace con energía, edad y hambre en cero.
- **Muerte**. Por inanición (superar $H_{\text{máx}} = 4$ ticks consecutivos sin comer) o por vejez (superar $A_{\text{máx}} = 60$ ticks). Un herbívoro depredado desaparece sin más.

2.5. Carnívoros

Análogos a los herbívoros, sin conducta de huida (no tienen depredador):

Prioridad	Condición	Acción
1	Hay un herbívoro adyacente	Cazar : moverse a esa celda y consumirlo (+2 energía, hambre a 0)
2	Hay una celda vecina vacía	Explorar : moverse a ella
3	Ninguna de las anteriores	Permanecer inmóvil

La reproducción sigue el mismo mecanismo ($E_{\text{repro}} = 3$). La muerte ocurre por inanición ($H_{\text{máx}} = 12$) o vejez ($A_{\text{máx}} = 90$). Los carnívoros toleran más hambre y viven más porque su recurso es más escaso y disperso.

2.6. Interacción entre especies

- **Depredación**. Cadena trófica lineal: plantas $\rightarrow$ herbívoros $\rightarrow$ carnívoros. Comer es siempre un movimiento: el depredador ocupa la celda de la presa.
- **Competencia por recursos**. Cuando varios agentes de la misma especie desean la misma celda destino, solo uno la obtiene; los demás permanecen en su posición y acumulan un tick de hambre. La competencia es, por tanto, *implícita* en la resolución de conflictos de movimiento (Sección 4).
- **Ciclo de retroalimentación**. Muchos herbívoros reducen la vegetación, lo que provoca inanición y una caída de la población de herbívoros; esto a su vez reduce el alimento de los carnívoros y permite la recuperación de las plantas. El resultado esperado son oscilaciones desfasadas de tipo Lotka–Volterra.

2.7. Población inicial

La población se define por **densidad** (fracción de celdas ocupadas), no por conteo absoluto, de modo que los parámetros sean independientes del tamaño de la cuadrícula. A cada celda se le asigna un valor $u \sim U[0, 1)$ y se decide por umbrales acumulados:

Rango de u	Contenido	Probabilidad
[0,00, 0,40)	Planta	40 %
[0,40, 0,50)	Herbívoro	10 %
[0,50, 0,52)	Carnívoro	2 %
[0,52, 1,00)	Vacío	48 %

La proporción 40 : 10 : 2 reproduce una **pirámide trófica** (20 plantas por carnívoro). Densidades iguales conducen al colapso: con un depredador por cada presa, los carnívoros agotan a los herbívoros en pocos ticks y mueren de inanición a continuación.

2.8. Parámetros

Todos los valores son configurables por línea de comandos. Los valores por defecto fueron calibrados experimentalmente para producir coexistencia estable durante más de 2000 ticks.

Parámetro	Opción	Herbívoro	Carnívoro
Energía ganada al comer	--ganh / --ganc	1	2
Energía para dejar cría	--reph / --repc	3	3
Ticks sin comer tolerados	--hamh / --hamc	4	12
Edad máxima	--edah / --edac	60	90

Para las plantas: $p_{\text{repro}} = 0,25$ (--prepro).

2.9. Supuestos declarados

El enunciado no especifica los siguientes puntos; se resolvieron así y se documentan por transparencia:

1. **Una entidad por celda.** El ejemplo de salida del enunciado muestra una única letra por celda.
2. **Frontera toroidal**, para uniformizar el vecindario.
3. **Destinos válidos:** un agente solo puede moverse a una celda que al inicio de la fase estuviera vacía o contuviera a su presa.
4. **Comer implica moverse:** un agente que permanece inmóvil no se alimenta.
5. **Reproducción solo al moverse**, de modo que la cría ocupe la celda de origen, garantizadamente libre.
6. **Vejez y umbrales de energía** no estaban cuantificados en el enunciado; se expusieron como parámetros.

3. El problema de paralelización

El enunciado propone el siguiente esquema:

```
Para cada tick:
    #pragma omp parallel for
    Para cada celda: actualizar plantas / herbívoros / carnívoros
    Sincronizar datos de especies entre hilos
```

Este esquema es incorrecto por tres razones:

1. Condición de carrera en el vecindario. Cada celda lee y escribe las celdas de sus 8 vecinos. Dos hilos que procesen celdas adyacentes escriben la misma memoria sin sincronización. En términos del modelo: dos herbívoros pueden consumir la *misma* planta, o dos agentes ocupar simultáneamente la misma celda destino.

2. Dependencia del orden de recorrido. Si el estado se actualiza sobre la misma cuadrícula que se lee, un agente que se desplaza “hacia adelante” vuelve a ser procesado en el mismo tick, actuando dos veces. En ejecución serial este sesgo es al menos determinista; en paralelo el orden depende del reparto de iteraciones, de modo que **el resultado cambiaría con OMP_NUM_THREADS**. Una simulación cuyo resultado depende del número de hilos no es verificable.

3. La sincronización no está especificada. La instrucción “sincronizar datos de especies entre hilos” no describe un mecanismo. Es precisamente la parte difícil del problema.

4. Estrategia de paralelización

La estrategia implementada es de **paralelismo de datos con descomposición en fases y arbitraje determinista**, conocida en la literatura de simulación basada en agentes como patrón *propose-commit*. Se apoya en tres mecanismos.

4.1. Doble buffer

Se mantienen dos cuadrículas: una **actual**, de solo lectura durante la fase, y una **siguiente**, de solo escritura. Al terminar la fase se permutan los punteros, operación en $O(1)$ que no copia datos.

Esto resuelve el defecto 2: todos los agentes observan el mismo estado inicial del tick, y ninguno puede actuar dos veces.

4.2. Fases secuenciales por especie

Cada tick se divide en tres fases consecutivas —plantas, herbívoros, carnívoros— separadas por un intercambio de buffers. El costo es recorrer la cuadrícula tres veces en lugar de una, pero cada fase presenta un patrón de dependencias mucho más simple, ya que solo una especie se desplaza a la vez.

4.3. Proponer, resolver, aplicar

Cada fase se descompone en tres barridos paralelos, separados por la barrera implícita del reparto de trabajo:

Etapa	Lee	Escribe
Proponer	cuadrícula actual	su propia entrada: a dónde desea moverse
Resolver	las propuestas	su propia entrada: quién la ocupará
Aplicar	todo lo anterior	su propia celda en la cuadrícula siguiente

La propiedad clave es que, en las tres etapas, **cada iteración escribe únicamente su propia posición**. Si ningún hilo escribe donde otro escribe, no puede haber condición de carrera. En consecuencia, **el núcleo de la simulación no contiene ninguna directiva critical, atomic ni omp_lock_t**.

Ejemplo. Dos herbívoros *A* y *B* adyacentes a la misma planta:

1. *Proponer*: *A* anota “deseo la celda central” con clave 8471; *B* anota lo mismo con clave 3092. Ninguno se ha movido.
2. *Resolver*: la celda central examina a sus vecinos, encuentra dos pretendientes y registra al de mayor clave: *A*.
3. *Aplicar*: la celda de *A* queda vacía (o con cría); la celda central pasa a contener a *A* con +1 energía; *B* permanece en su posición y suma un tick de hambre.

4.4. Inversión del patrón de escritura: *gather* en lugar de *scatter*

El problema en su formulación natural es un *scatter*: varios agentes compiten por escribir en una misma celda destino, lo que exigiría exclusión mutua.

La observación que lo resuelve es que **toda propuesta apunta a una celda adyacente**. Por lo tanto, los únicos candidatos posibles a ocupar una celda dada son sus 8 vecinos. Basta entonces invertir la dirección: en lugar de que los agentes escriban sobre el destino, es el destino quien examina a sus vecinos y elige. Esto convierte el *scatter* en un *gather*, y el conflicto desaparece sin ningún mecanismo de sincronización.

4.5. Arbitraje determinista

Cada propuesta lleva una clave pseudoaleatoria $k = \text{hash}(\text{semilla}, \text{tick}, \text{fase}, \text{origen})$. Gana la clave mayor; en caso de empate, el índice de origen menor. Como la clave no depende del hilo ni del orden de iteración, el ganador es siempre el mismo.

Esta propiedad es la que permite **verificar la corrección**: si el resultado con 8 hilos coincide byte a byte con el de la versión serial, entonces la paralelización no introdujo errores.

4.6. Generación de números aleatorios sin estado

La función `rand()` es inadecuada aquí por dos razones. Primero, no es *thread-safe*: en glibc emplea un cerrojo interno que serializa la región paralela. Segundo, y más grave, aunque se usara un generador por hilo, el valor consumido por una celda dependería de *qué hilo la procesó* y de cuántas celdas hubiera procesado antes, de modo que el resultado volvería a depender del número de hilos.

La solución adoptada elimina el estado: el valor aleatorio de una celda es una **función hash pura** de (semilla, tick, fase, flujo, índice). No hay variable compartida que proteger, no hay contención, y el valor correspondiente a una celda concreta es siempre el mismo. Es además más rápido que `rand()` incluso en ejecución serial.

5. Conceptos de OpenMP aplicados

5.1. Directivas y cláusulas utilizadas

Elemento	Ubicación	Uso
<code>parallel</code>	<code>sim.c</code>	Una única región que envuelve el bucle de ticks completo
<code>for</code>	<code>fases.c</code>	Nueve barridos por tick (3 etapas $\times$ 3 fases)
<code>single</code>	<code>sim.c</code>	Intercambio de buffers y salida por pantalla
<code>atomic</code>	<code>sim.c</code>	Únicamente en el censo de población: 3 por hilo por tick
<code>private</code>	<code>fases.c</code>	<code>private(v)</code> : arreglo de 8 vecinos, propio de cada hilo
<code>nowait</code>	<code>sim.c</code>	Elimina una barrera innecesaria tras el censo
<code>schedule(runtime)</code>	<code>fases.c</code>	Permite variar la política de reparto sin recompilar
<code>omp_get_wtime</code>	<code>sim.c</code>	Medición de tiempos

5.2. Región paralela única y directivas huérfanas

La región `parallel` envuelve el bucle de ticks completo, no cada bucle interno. Con T ticks, 3 fases y 3 barridos por fase, la versión ingenua pagaría $9T$ creaciones y destrucciones de equipo de hilos; esta implementación paga exactamente una. La coordinación entre etapas se logra con las barreras implícitas de `omp for` y `omp single`, sensiblemente más baratas.

Para conseguirlo sin sacrificar la modularidad, las funciones de `fases.c` emplean **directivas** `omp for` **huérfanas**: no tienen un `omp parallel` sintácticamente por encima, sino que se ligan en tiempo de ejecución a la región abierta en `sim.c`.

5.3. Prevención de *false sharing*

El censo de población se implementó como una reducción manual: cada hilo acumula en variables *locales* y suma al final con tres operaciones atómicas. La alternativa aparentemente natural — un arreglo `contador[id_hilo]` — colocaría todos los contadores en la misma línea de caché; cada incremento invalidaría la línea en los demás núcleos, degradando el rendimiento de forma severa sin producir ningún error de corrección.

5.4. Elementos deliberadamente no utilizados

Elemento	Razón
<code>critical</code>	El <i>gather</i> lo vuelve innecesario
<code>reduction</code>	Se implementó manualmente para hacer explícita la prevención de <i>false sharing</i>
<code>sections</code>	Es paralelismo de tareas; el problema es de datos. Las tres fases son dependientes entre sí
<code>flush</code>	Las barreras implícitas de <code>for</code> y <code>single</code> ya implican <i>flush</i>
Paralelismo anidado	Un solo nivel es suficiente; anidar solo añadiría sobrecarga

6. Decisiones de implementación

Decisión	Justificación
Celda de 8 bytes, sin punteros	Caben 8 celdas por línea de caché; el arreglo contiguo maximiza la localidad. Una lista de agentes enlazada destruiría el patrón de acceso.
Cuadrícula aplanada a 1D	Recorrido en orden de memoria; el índice es la unidad natural de reparto.
Frontera toroidal	Elimina los casos especiales de borde y garantiza 8 vecinos por celda.
Inicialización en paralelo	Aplica <i>first-touch</i> : cada hilo escribe primero las páginas que después procesará, lo que en máquinas NUMA sitúa los datos en el banco de memoria correcto.
<code>schedule(runtime)</code>	Permite comparar políticas de reparto con <code>OMP_SCHEDULE</code> sin recompilar.
Población por densidad	Los parámetros son independientes del tamaño de la cuadrícula.
Salida desactivable (<code>--cada 0</code>)	La impresión es totalmente serial; dejarla activa contaminaría toda medición de rendimiento.
Muestreo por reservorio	Elige uniformemente entre los vecinos válidos en una sola pasada, sin arreglo auxiliar ni conocer de antemano cuántos hay.

Restricción $W, H \geq 3$. La etapa *resolver* se apoya en que la relación de vecindad sea simétrica: s es vecino de d si y solo si d es vecino de s . En un toro de anchura 2, las columnas $x - 1$ y $x + 1$ coinciden, el vecindario repite índices y esa simetría se pierde. El programa valida y rechaza esas dimensiones.

7. Verificación de corrección

El determinismo del diseño habilita una prueba de corrección poco frecuente en código paralelo: comparar la salida contra la versión serial.

1. **Invarianza al número de hilos.** La salida con 1, 2, 3, 5, 8 y 16 hilos debe ser idéntica.
2. **Invarianza a la política de reparto.** Igualmente para `static`, `static,1`, `dynamic,1`, `dynamic,64` y `guided`.
3. **Coincidencia con la compilación serial.** El mismo código compilado `sin -fopenmp` debe producir la misma salida.

Un error de sincronización que sobreviviera a 16 hilos con `dynamic,1` durante miles de ticks sin producir una sola divergencia es estadísticamente inverosímil; por ello esta prueba es más concluyente que un analizador estático.

Nota sobre ThreadSanitizer. La biblioteca `libgomp` de GCC no está anotada para ThreadSanitizer, que por tanto no reconoce las barreras implícitas de `omp for` ni la sincronización al cerrar la región paralela: interpreta como carrera toda escritura de un hilo seguida de la lectura de otro a través de una barrera. Es el mismo problema, agravado, que hace poco útiles a Helgrind y DRD de Valgrind con OpenMP. La herramienta adecuada es **Archer**, la capa que anota el *runtime* de OpenMP para ThreadSanitizer, disponible con Clang y la `libomp` de LLVM.

8. Análisis de resultados

A continuación, se presentan las mediciones de rendimiento y el análisis del comportamiento poblacional obtenidos tras ejecutar las distintas simulaciones en el ecosistema.

8.1. Entorno de pruebas

Procesador	Intel Core i7-10700K, 8 núcleos físicos / 16 lógicos
Memoria	16 GB
Sistema	Windows 11
Compilador	GCC 6.3.0 (MinGW), OpenMP 4.5
Banderas	-std=c11 -O2 -fopenmp
Metodología	Mediana de 3 corridas por configuración; salida desactivada

8.2. Escalabilidad fuerte

Cuadrícula fija, número de hilos variable. $S_n = T_1/T_n$ (*speedup*), $E_n = S_n/n$ (eficiencia).

Hilos	Tiempo (s)	Speedup	Eficiencia
1	13.01	1.00	1.00
2	9.03	1.44	0.72
4	5.69	2.29	0.57
6	5.05	2.58	0.43
8	4.58	2.84	0.36

La curva de *speedup* muestra un crecimiento inicial, pero se aplana notablemente al acercarse a los 8 hilos. Como se esperaba, la eficiencia cae de 1.00 a 0.36, demostrando que el algoritmo, al realizar poca aritmética por byte leído (lectura de 8 vecinos, escritura de una celda), satura rápidamente el ancho de banda de memoria (memory-bound), limitándose por este factor y no por la pura capacidad de cómputo del procesador.

8.3. Políticas de reparto de carga

OMP_SCHEDULE	Tiempo (s)	Observación
static	3.45	Línea base
static,64	3.41	Ligera mejora sobre static por mejor localidad de caché
dynamic,1	14.72	Muy lento por alta sobrecarga de coordinación
dynamic,64	3.92	Buen balance, supera ampliamente a dynamic,1
guided	3.08	El mejor rendimiento obtenido

La carga por celda es desigual: una celda vacía se copia sin más, mientras que un carnívoro rodeado de presas recorre su vecindario varias veces. Como se hipotetizó, la política **guided** fue la más rápida (3.08 s), adaptándose armónicamente a la carga irregular. En contraste, **dynamic,1** (14.72 s) resultó ser incluso más lento que la ejecución serial (13.01 s) debido al inmenso costo de coordinación de solicitar iteraciones una a una.

8.4. Fracción serial y ley de Amdahl

Configuración	Tiempo (s)	Fracción serial
Sin salida (<code>--cada 0</code>)	1.54	—
Con salida (<code>--cada 1 --grid</code>)	25.50	$\approx 94\%$

La abismal diferencia corresponde al costo de la impresión, que se ejecuta en un único hilo de forma secuencial. Al representar aproximadamente el 94% del tiempo total en ejecuciones con salida visual, la fracción serial f restringe drásticamente el speedup máximo alcanzable a $1/f$, con independencia de si se añaden cientos de hilos.

8.5. Sobresuscripción

Al ejecutar la simulación forzando 32 hilos (superando los 16 hilos lógicos del sistema), el tiempo se degradó a 6.85 s frente a los 4.58 s obtenidos con 8 hilos. Esta pérdida de rendimiento ilustra a la perfección el impacto negativo de los constantes cambios de contexto impuestos por el sistema operativo al intentar intercalar una sobrepoblación de hilos.

8.6. Dinámica del ecosistema

En la gráfica de las tres poblaciones frente al tiempo, sobre una corrida larga (~ 1000 ticks), se observa un transitorio inicial de ~ 50 ticks debido al ajuste desde la distribución aleatoria de partida. Posteriormente, se establecen hermosas oscilaciones con un claro desfase entre los picos de herbívoros y carnívoros, reproduciendo con éxito la dinámica clásica depredador-presa.

8.7. Sensibilidad a la población inicial

La comparación entre la pirámide trófica (40 : 10 : 2) y densidades iguales (33 : 33 : 33) demuestra cómo la primera logra una coexistencia sumamente estable y prolongada, mientras que el segundo escenario documenta un rápido colapso ecológico debido a la sobreexplotación desmedida por parte de los carnívoros, seguida por su eventual inanición.

9. Conclusiones

- La reformulación del conflicto de escritura, transformándolo magistralmente en un enfoque *gather*, permitió una implementación paralela sumamente fluida y totalmente libre de sincronización bloqueante.
- El determinismo, obtenido mediante un generador aleatorio puro sin estado y un arbitraje independiente del orden, hizo posible verificar la corrección del modelo contrastándolo de forma idéntica contra la ejecución serial.
- La experimentación confirmó que la elección de la política de reparto es puramente un parámetro de ajuste visual y de rendimiento (donde `guided` brilló), demostrando que la estrategia base de paralelización funciona de manera robusta e independiente.

Anexo: instrucciones de compilación y ejecución

```
gcc -std=c11 -O2 -Wall -Wextra -fopenmp ^
    src/main.c src/mundo.c src/fases.c src/sim.c -o ecosim.exe -lm

.\ecosim.exe --w 60 --h 30 --ticks 200 --cada 20 --grid --log resultados.log
.\ecosim.exe --ayuda
```

Estructura del código fuente:

src/ecosim.h	Tipos, configuración, prototipos
src/rng.h	Generador determinista por celda
src/mundo.c	Memoria, inicialización, intercambio de buffers, salida
src/fases.c	Núcleo: proponer, resolver, aplicar
src/sim.c	Bucle de ticks y región paralela
src/main.c	Línea de comandos y medición de tiempos
